BLE-Based Hiker Monitoring — Stage D: Cross-Sensor Identity Linking & Direction

Research report | Updated August 1, 2026 (rev. 2: loss fix + transit-gate sweep) | Stages A–D complete (D: implemented, calibrated, honestly limited)

Contents

1. Background and objective
2. Stages A–C: data, method and results (dedup, sessionization, entity resolution) + Stage D preprocessing
3. Motivating findings from ground truth
4. Method
5. Calibration procedure and results
6. Two follow-up experiments: loss fix and transit-gate sweep
7. Applying the calibrated matcher to all dual-sensor dates
8. Why two ground-truth dates are structurally unreachable
9. Honest assessment: does direction detection work?
10. Conclusions and recommended next steps

7,961 / 10,516
unique travelers (A / B) after Stages A–C

30
collection days, 14 dual-sensor — the base for Stage D

5,964
pairs linked by Stage D across all 14 dual-sensor dates

0.490
direction-share correlation vs. manual ground truth (not significant, n=8)

What changed in this revision: per follow-up discussion, two things were tested — (1) the calibration loss no longer over-weights volume vs. direction, and (2) the transit-time gate `max_transit_s` is now fitted instead of fixed at 300s. Result: honestly mixed (Section 6), not a breakthrough. Separately, the two structurally-unreachable ground-truth dates were diagnosed to have two *different* root causes (Section 8), and by decision, Stage C is left unmodified rather than retuned against these same 8 windows.

1. Background and objective

This project develops a low-cost, passive method for counting hikers on outdoor trails using Bluetooth Low Energy (BLE) advertisement sniffing from two Arduino-based sensors, **Sensor A** and **Sensor B**, placed a short distance apart (~20m) along the same trail with an overlapping detection range. Stages A–C (documented separately) turned raw BLE sniffs into a deduplicated, sessionized, per-sensor `Traveler_ID` table — but did so **independently per sensor**. Neither stage ever connected a traveler's Sensor A presence to their Sensor B presence.

Stage D's job: link travelers across the two sensors and derive a walking **direction** (A→B or B→A) from the link. This report documents the project **end to end** — the method, decisions and results of Stages **A (deduplication), B (sessionization), and C (entity resolution)** in summary (Section 2), followed by **Stage D**'s method in depth — the decisions behind it, the calibration against the project's only ground truth, and — because the honest result matters more than a polished-looking pipeline — exactly how well (and how poorly) the result actually works.

2. Data and preprocessing recap (Stages A–C)

Stage D consumes Stage C's output directly: `sessions_sensor_A/B_with_traveler` (one row per contiguous "session" of a MAC address, tagged with a `Traveler_ID`) plus `DATA/combined_file_manual.csv`, a human observer's manual click-counter log — the project's only ground truth, covering 8 short windows across 8 distinct dates (17–237 people each).

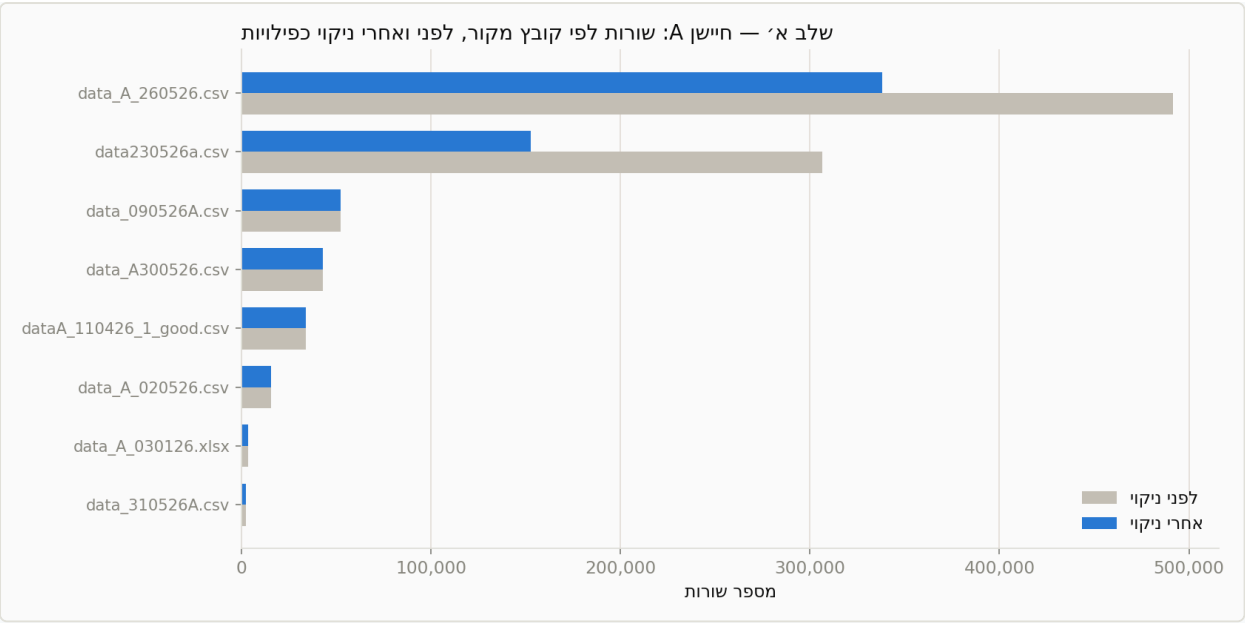
Stage	What it does	Output consumed by Stage D
A — Deduplication	Removes exact-duplicate raw readings (cumulative-export artifact)	indirectly, via B/C
B — Sessionization	Groups raw readings into contiguous per-MAC "sessions" (60s gap threshold), computing `mean_rssi` and `rssi_trend` per session	`sessions_sensor_A/B`
C — Entity resolution	Merges MAC addresses likely belonging to the same physical traveler into one `Traveler_ID` (per sensor, independently)	`sessions_sensor_A/B_with_traveler`
D — This stage	Links Sensor-A travelers to Sensor-B travelers and derives direction	—

The data arrives as separate export files per sensor, collected over different periods: **15** source files in total (8 for Sensor A, 7 for Sensor B), covering **30** distinct collection days (21 for Sensor A, 23 for Sensor B, overlapping on 14 shared days — the only data Stage D can ever run on).

Stage A — Deduplication

Source files are sometimes a **cumulative export** of the same device log, so naively concatenating all files creates exact duplicates of the same reading. Cleaning defines an identity key per reading — the triple `(MAC, RSSI, timestamp)` — and drops any row whose key already appeared earlier in the combined set. A three-column key (not just MAC and time) is deliberate: two back-to-back readings of the same MAC in the same second with *different* RSSI are genuine distinct readings (natural BLE-scan noise), not export duplicates.

sensor	rows before cleaning	rows after cleaning	duplicates removed	% removed
A	949,211	641,804	307,407	32.4%
B	1,217,216	815,093	402,123	33.0%

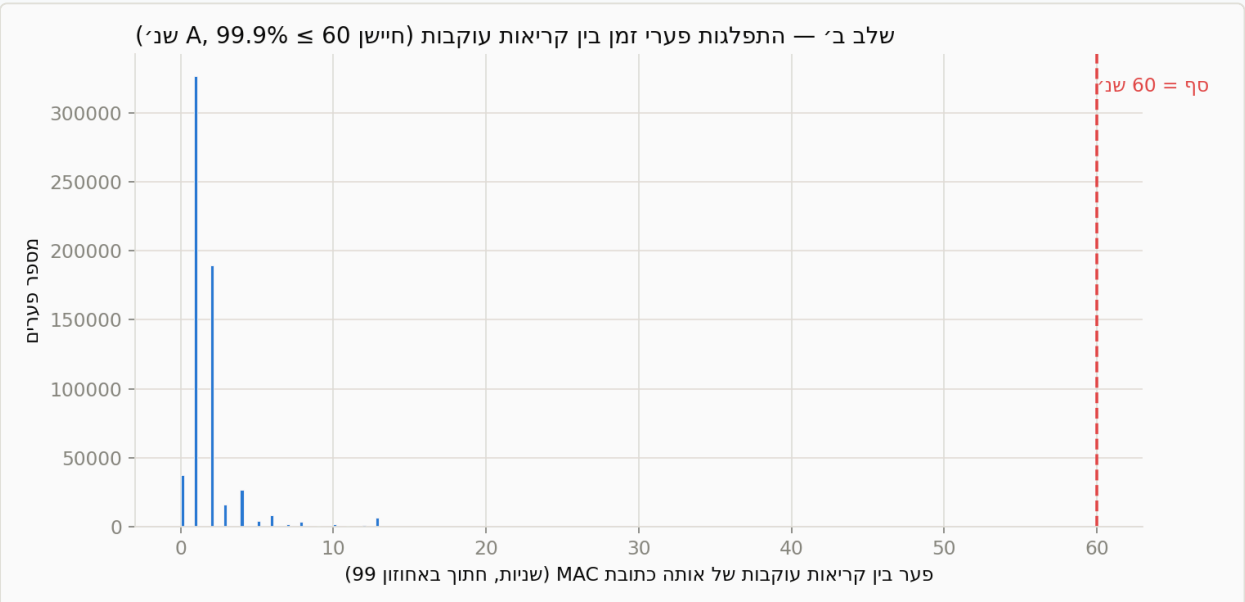


Rows per source file (Sensor A), before and after cleaning. The largest files lose the most rows — consistent with being a cumulative export of the same log.

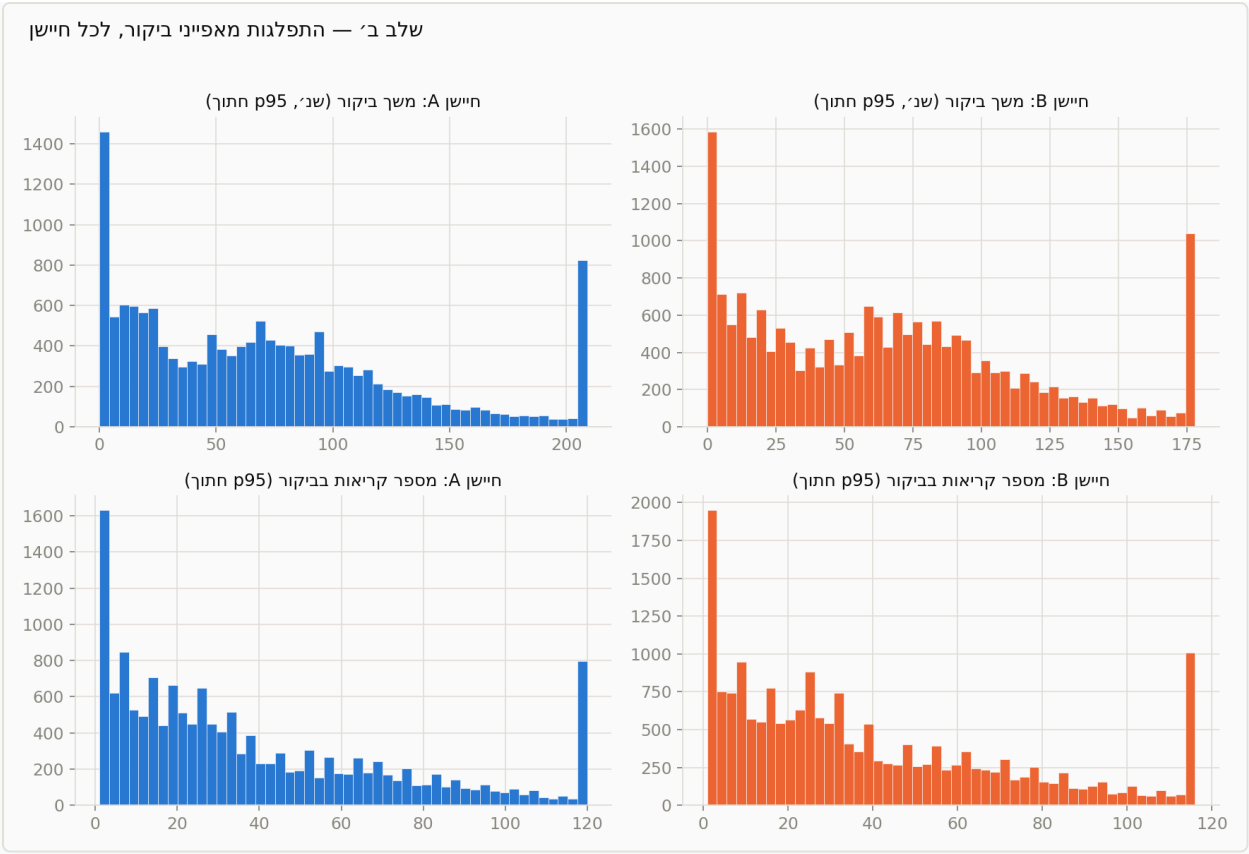
Stage B — Sessionization

Raw RSSI readings of the same MAC address, per sensor, are grouped into contiguous "sessions": whenever the gap between two consecutive readings of the same MAC exceeds a **60-second threshold** (derived empirically — 99.9% of gaps are at or below it), a new session starts. Each session gets a start/end time, mean RSSI, and **RSSI trend** — the slope of a linear regression of RSSI against time, in dB/s — which is the feature Stage D's `rssi_shape_score` is built on (Section 2 below).

sensor	readings (post-cleaning)	sessions	single-reading sessions	multi-reading sessions
A	641,804	15,267	649 (4.3%)	14,618
B	815,093	19,359	801 (4.1%)	18,558



Distribution of gaps between consecutive readings of the same MAC address. The threshold line (60s, red) sits far out in the tail.



Distribution of session duration and reading count per session, per sensor — very similar shapes between A and B.

> **How the sessionization algorithm works, step by step:** for each MAC address separately — (1) sort all its readings by time; (2) walk through the readings in order, measuring the time gap from the previous reading; (3) whenever the gap exceeds 60 seconds, "close" the current session and open a new one; (4) every run of consecutive readings produced this way is one session. This is a simple special case of **single-linkage clustering** in a single dimension (time only): a reading "joins" a session if it's close enough in time to the reading immediately before it — no need to compute a distance between every possible pair of readings, only between neighbors in chronological order, which is why it stays fast even on millions of rows.

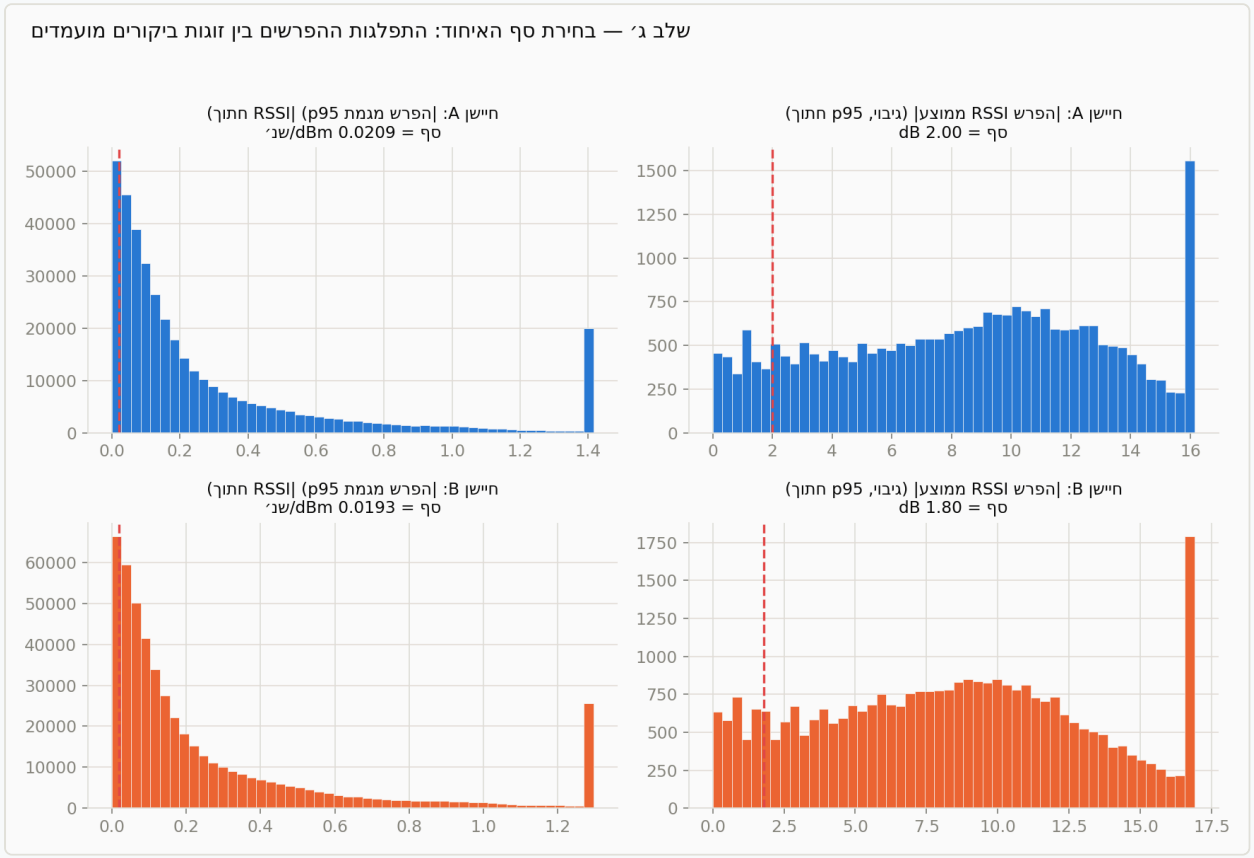
Stage C — Entity resolution

Distinct MAC addresses likely belonging to the same physical traveler (due to carrying multiple devices, or a mid-visit MAC change) are merged under one `Traveler_ID`, separately per sensor. Sessions are first blocked into contiguous time blocks, then grouped within each block using **Complete-Linkage** hierarchical clustering on two criteria — overlapping time (or a gap under 60s) **and** a small difference in RSSI trend (with a mean-RSSI fallback for single-reading sessions). The similarity threshold is set at the **10th percentile** of the difference distribution across candidate pairs — a conservative choice, since there's no ground truth to calibrate a more precise threshold. This is exactly the "no ground truth" limitation that Stage D partially solves using the manual count log (Section 3 below).

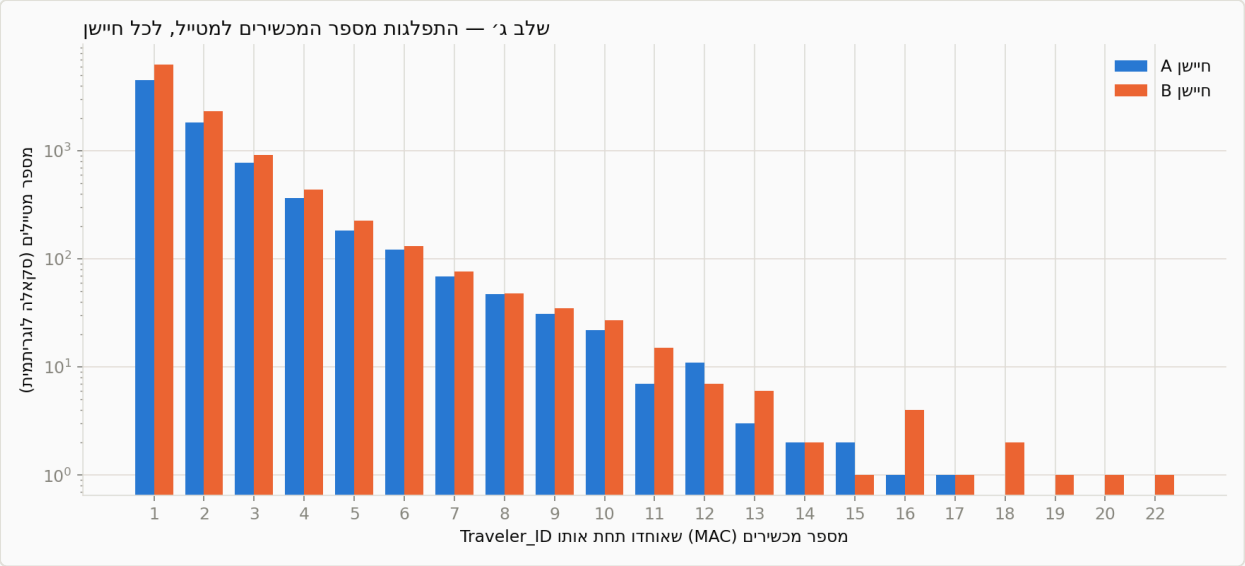
> **How the Complete-Linkage clustering algorithm works, step by step:** within each time block — (1) compute the difference (in RSSI trend, or mean RSSI for single-reading sessions) between every pair of sessions in the block; (2) start with every session as its own singleton cluster; (3) at each step, find the two clusters whose **maximum** pairwise distance between any of their sessions (not the average, not the minimum) is smallest, and merge them; (4) repeat, as long as that maximum distance for the candidate

merge stays below the threshold (10th percentile). Result: a cluster only forms if *every* session inside it — including the most "distant" pair — is still similar enough. That's the key difference from **single-linkage** (used in Stage B above): there, it's enough for two *chain-adjacent* sessions to be similar to connect, even if the two ends of the chain are nothing alike — which can chain-merge a cluster that isn't actually similar all the way through.

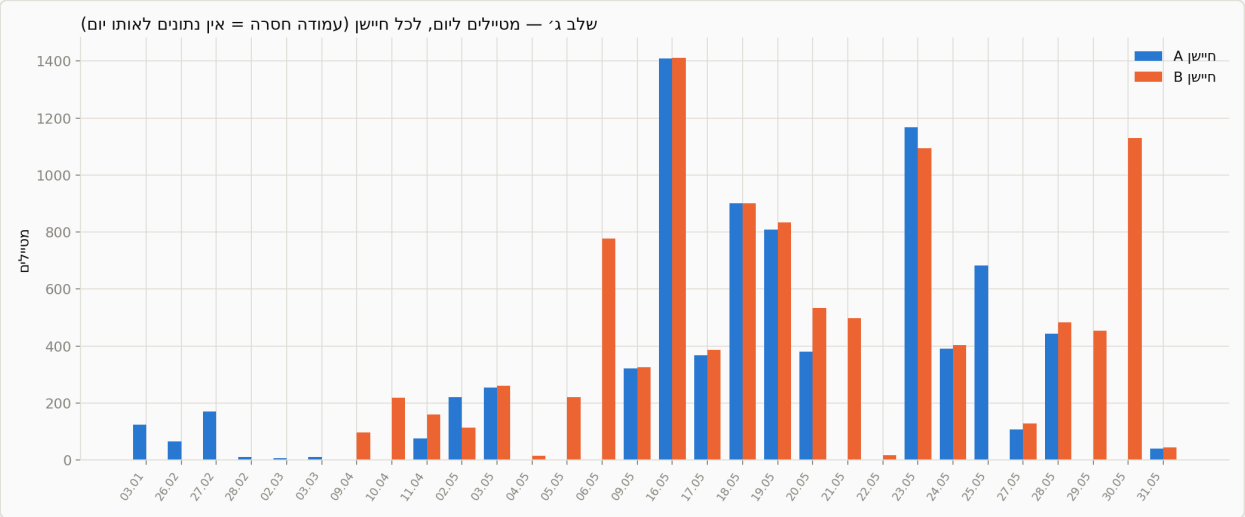
sensor	sessions	travelers (Traveler_ID)	merges performed	largest cluster
A	15,267	7,961	7,306	17 devices
B	19,359	10,516	8,843	22 devices



Distribution of differences between candidate session pairs, per sensor and per criterion. The threshold line (dashed red) is the 10th percentile.



Distribution of the number of devices merged under the same traveler (log scale). ~56–60% of travelers were identified by a single device alone.

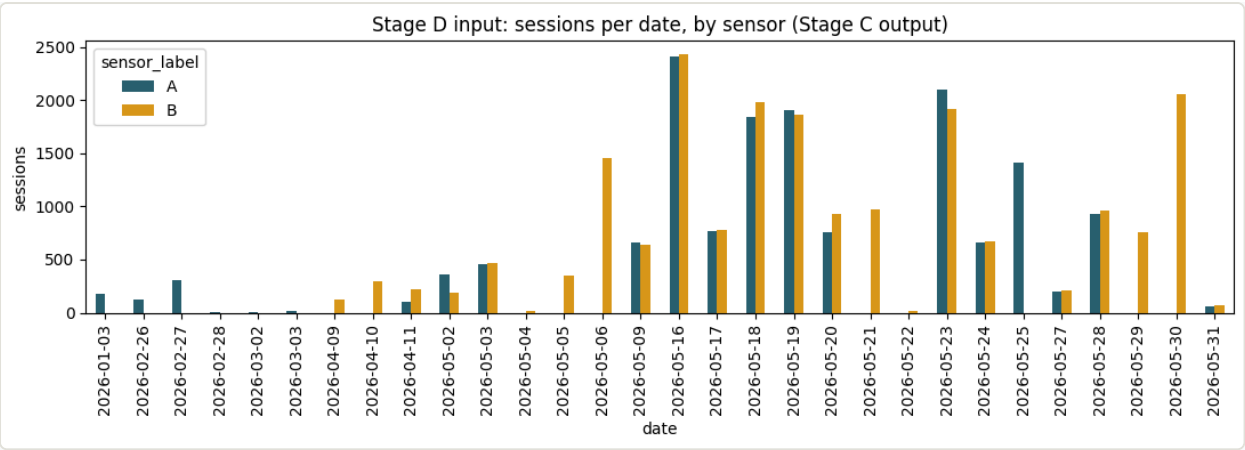


Unique travelers identified per collection day, per sensor. On the 14 shared days, the two sensors' counts — computed by fully independent pipelines — track closely, indirect evidence of methodological stability.

> **Why this matters for Stage D:** Stage D directly inherits Stages A–C's "no ground truth" limitation — and their outputs. In particular, Stage C's "compression" rate (sessions → unique travelers) is exactly what sets the structural ceiling on how many pairs Stage D can ever match (Section 5.2), and its over-merging on 2026-05-19 is one of the two reasons that ground-truth date is structurally unreachable (Section 8).

What the Stage D input looks like ("before")

Each row of the Stage C output is one session: a `Traveler_ID`, a time window (`start_time` / `end_time`), and two RSSI summary features (`mean_rssi`, `rssi_trend`, in dB and dB/s). A single traveler typically has several sessions across a visit.



Sessions per date, by sensor — the raw input volume Stage D works from, before any cross-sensor linking.

From raw sessions to comparison anchors ("after")

Stage D does not compare travelers' full visit spans. It compares the single session **closest to the boundary** with the other sensor: a Sensor-A traveler's **last** session (about to leave A's range) against a Sensor-B traveler's **first** session (just entering B's range), and symmetrically for the reverse direction. This transformation — many raw sessions down to two boundary "anchors" per traveler — is the key preprocessing step specific to Stage D.

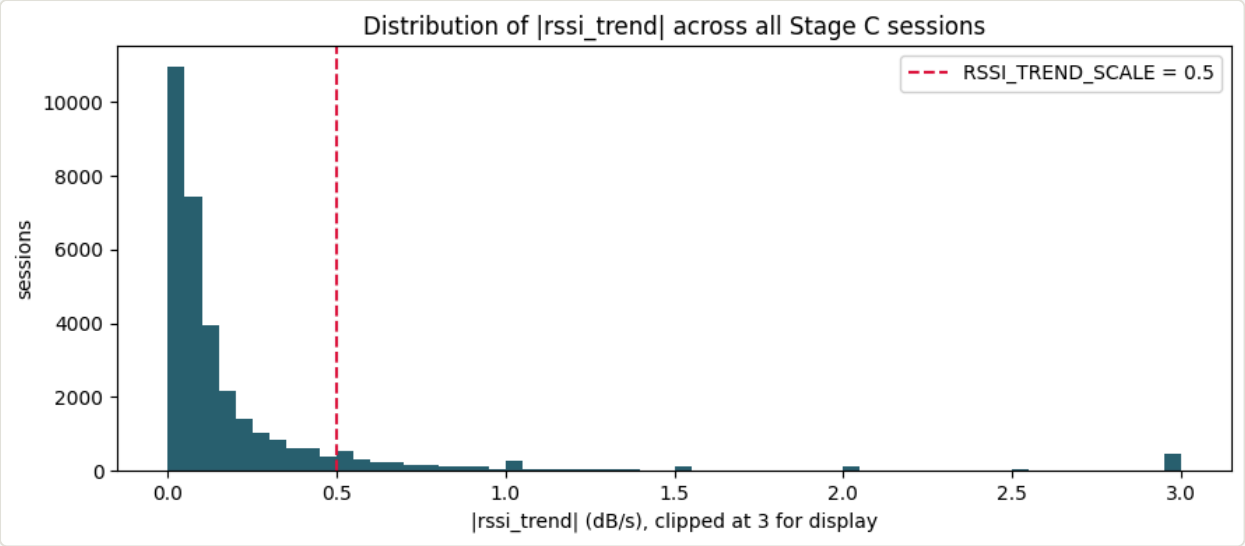
Worked example from the executed notebook — Sensor A traveler `A-50-1` on 2026-04-11:

BEFORE — raw sessions for this traveler				
Traveler_ID	**start_time**	**end_time**	**mean_rssi**	**rssi_trend**
A-50-1	2026-04-11 09:05:32	2026-04-11 09:06:51	-85.09	0.086
A-50-1	2026-04-11 09:06:00	2026-04-11 09:06:30	-85.77	0.084

AFTER — the two boundary anchors Stage D compares against Sensor B			
anchor	**time**	**mean_rssi**	**rssi_trend**
first_session_anchor (entry side)	2026-04-11 09:05:32	-85.09	0.086
last_session_anchor (exit side)	2026-04-11 09:06:51	-85.09	0.086

Empirical basis for the RSSI-trend saturation scale

`rssi_shape_score` saturates once `|rssi_trend|` reaches `RSSI_TREND_SCALE = 0.5` dB/s. Checked directly against the real session data: median 0.085, p75 0.209, p90 0.524, p95 0.942 dB/s — so 0.5 sits around the 90th percentile, letting a clearly unambiguous fade/rise saturate the score instead of letting rare, very steep trends dominate.



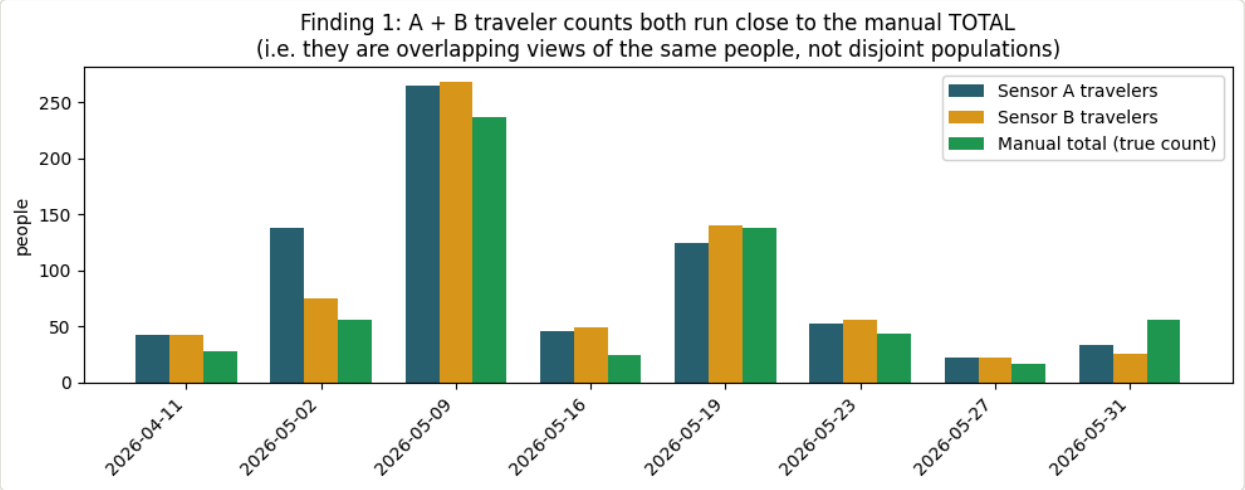
Distribution of |rssi_trend| across all Stage C sessions, both sensors combined, with the RSSI_TREND_SCALE=0.5 cutoff marked.

3. Motivating findings from ground truth

Before designing the matching algorithm, the manual ground-truth data was used to test three naive approaches. All three failed, for identifiable reasons that directly shaped the design below.

Finding 1 — Summing each sensor's independent count double-counts

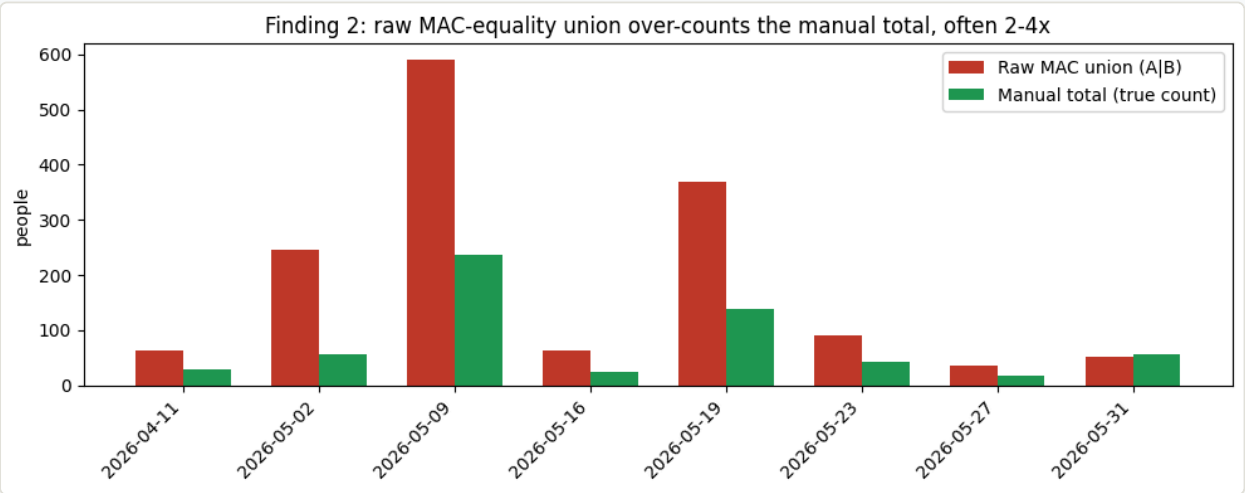
Cross-checking traveler counts in each ground-truth window shows Sensor A's and Sensor B's counts both run close to the manual **total**, not to disjoint halves of it — consistent with the ~20m sensor gap and a ~10–15s walking transit at a typical 5–6 km/h pace. The two sensors are mostly-overlapping views of the *same* people, not disjoint populations.



Finding 1: Sensor A and Sensor B traveler counts per ground-truth window, both plotted against the manual total.

Finding 2 — Raw MAC-address matching over-counts

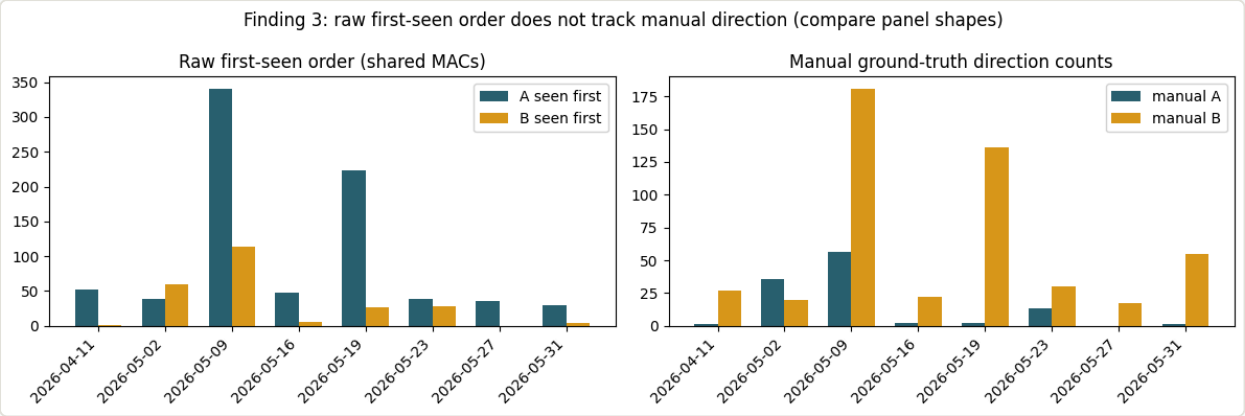
Matching Sensor A and B readings by literal MAC equality within each ground-truth window produces union totals 2–4x the manual ground truth in most windows — worse than either sensor's own already-inflated count. MAC randomization mid-transit and MAC-level (rather than trip-level) matching both amplify overcounting when used naively.



Finding 2: raw MAC-equality union vs. manual total, per ground-truth window.

Finding 3 — "First sensor to see the MAC" is not a direction signal

Comparing which sensor first detected a shared MAC against the manual direction split shows the **opposite** skew from ground truth on multiple dates. This is consistent with Sensor A having a longer/more sensitive detection radius than Sensor B, so it systematically "sees" devices earlier *regardless of true walking direction* — a range-asymmetry confound, not a direction signal.



Finding 3: raw first-seen order (left) does not track the manual ground-truth direction split (right) — compare panel shapes per date.

> These three findings directly shape the method: match at the **traveler** level (not raw MAC) using signal **shape** (not MAC equality) at the **boundary** of each sensor's view (not first-ever-seen), and calibrate against the only ground truth available rather than assume a threshold.

4. Method

4.1 Candidate pairs and features

Within each date, every Sensor-A traveler is compared against every Sensor-B traveler. For each candidate pair, two features are computed:

- `transit_gap_s` : seconds between the two boundary anchors (A's-last→B's-first, or B's-last→A's-first — whichever is positive). This also fixes the candidate's direction. A hard physical gate rejects gaps outside `(0, max_transit_s]` seconds outright. Originally fixed at 300s — a walking speed of 0.067 m/s, an order of magnitude of slack over "someone dawdled" — later fitted instead of assumed (Section 6); the calibrated value is 180s.
- `rssi_shape_score` : a bounded `[0, 1]` agreement score between "fading out at the departure sensor" and "strengthening at the arrival sensor," computed as the geometric mean of each side's rectified, scale-clipped trend. Zero unless **both** anchors behave as a real transit would.

4.2 Matching: optimal bipartite assignment

Per date, a bipartite graph (Sensor-A travelers × Sensor-B travelers) is solved with `scipy.optimize.linear_sum_assignment` — optimal assignment rather than greedy nearest-match, since greedy can lock in a suboptimal pair and block a better one. Edges scoring below `min_match_score` are **forbidden before** the assignment is solved (not filtered afterward), so the optimizer cannot waste a traveler on a pairing it will discard. Travelers left unmatched are reported as "seen by only one sensor," not forced into a low-confidence match.

> **How the optimal-assignment algorithm actually works:** build a "cost" matrix sized (Sensor-A travelers) × (Sensor-B travelers), where each cell holds the cost of pairing that particular A-traveler with that particular B-traveler (in practice, minus the match score, turning "maximize score" into "minimize cost"). Forbidden pairs (below `min_match_score` , or outside the transit gate) get infinite cost, so the solver can never pick them. `scipy.optimize.linear_sum_assignment` — an implementation of the **Hungarian algorithm** — solves the problem exactly: it finds the set of pairs (at most `min(n_A, n_B)` , with no traveler shared between two pairs) that minimizes the **total cost summed across every pair at once** — not just the best score pair-by-pair. That's the difference from a **greedy** approach: greedy would match the highest-scoring pair first, then keep going with whatever is left — but an early pick like that can lock a traveler into a mediocre match and block a better pairing that would only become visible later. The optimal assignment looks at the whole picture simultaneously and guarantees no alternative assignment could have scored higher overall for that date.

4.3 Direction and score scale

`match_score = transit_time_score + rssi_weight × rssi_shape_score` , both terms bounded to `[0, 1]` , so the maximum attainable score is `1.0 + rssi_weight` . Direction is whichever anchor timestamp is earlier — no direction is assigned to unmatched travelers.

> **A bug caught and fixed during review:** an initial implementation scored `rssi_shape_score` as an **unbounded sum** of the two anchors' rectified trend magnitudes rather than a bounded similarity. With real `rssi_trend` reaching ±16 against a timing term capped at 1.0, and a calibrated `rssi_weight` of 5.0, RSSI steepness alone decided every match and timing became irrelevant — 18.2% of the shipped rows had a *negative* `transit_gap_s` (median ~88 minutes backwards). This was fixed (bounded `[0,1]` agreement

> score, hard 300s gate, forbid-before-not-filter-after thresholding, label orientation re-fitted per CV fold) and the calibration re-run. All results in this report are from the **corrected** implementation.

5. Calibration procedure and results

5.1 What was fitted, and how

Four grid parameters (`transit_center_s` , `transit_spread_s` , `rssi_weight` , `min_match_score`) plus a binary label orientation (`flip` — the manual log's A/B convention is undocumented, so which physical direction it means is itself an inference). Fitting used **leave-one-date-out cross-validation** across the 8 ground-truth dates: both the grid parameters *and* the orientation are re-fitted per fold from the other 7 dates only, so no held-out date's own ground truth leaks into its own evaluation.

5.2 Structural ceiling — is the target even reachable?

A bipartite matching can produce at most `min(n_A, n_B)` pairs. Two of the eight ground-truth dates have a manual total **larger** than this ceiling:

date	n_A travelers	n_B travelers	max possible matches	manual total
2026-05-19	124	140	124	**138**
2026-05-31	33	25	25	**56**

On those dates no parameter setting can reach the target — Stage C simply did not resolve enough distinct travelers on those days for the manual count to be reachable. This is a Stage A–C recall issue surfacing here, not a Stage D matching defect, but it does anchor part of the calibration loss to irreducible error.

5.3 Calibrated parameters (current, after the Section 6 fixes)

Grid: 7,560 points over `transit_center_s` ∈ {10…50}, `transit_spread_s` ∈ {8…60}, `rssi_weight` ∈ {0.0…2.0}, `min_match_score` ∈ {0.05…1.1}, and — as of this revision — `max_transit_s` ∈ {60, 90, 120, 180, 300}s, fitted rather than fixed (rationale in Section 6).

parameter	fitted value
`transit_center_s`	10.0
`transit_spread_s`	60.0
`rssi_weight`	**0.0**
`min_match_score`	0.05
`max_transit_s`	**180.0** (interior grid value, not a boundary hit)
label orientation	`flip = True`
in-sample total loss	8,957 (new loss function — not comparable in magnitude to the old 12,412)
mean held-out loss	1,288.9

> `rssi_weight` **still calibrates to 0.0** overall — the RSSI-shape feature earns zero weight against timing on the full 8-date fit. (One individual leave-one-out fold, holding out 2026-05-19, did find `rssi_weight=0.5`

> useful when fit on the other 7 dates — the "RSSI adds nothing" reading is the dominant pattern, not a universal one.)

Unlike the other four parameters, `max_transit_s` landed on an **interior** grid value (180s, not the endpoints 60s or 300s) — a genuine preferred setting, not another saturation-plateau artifact. Probing explicitly out-of-grid settings for the other four still shows the same plateau as before, with one honest caveat: with the corrected loss, a fully-flat probe now scores marginally *better* than the fitted grid corner (8,897 vs. 8,957, a ~0.7% difference, and per-date predicted totals shift by at most 1 person) — so this is closer to "the grid corner is good enough" than "provably optimal," a minor downgrade in confidence from the original run's cleaner plateau finding.

6. Two follow-up experiments: loss fix and transit-gate sweep

Two changes were tested on top of the original design, both aimed directly at the direction-skew problem:

1. **Loss fix.** The original `calibration_loss` summed squared error on `A_to_B`, `B_to_A`, **and** their sum `total`. Writing `a`, `b` for the two directional errors, that is `a² + b² + (a+b)² = 1.5(a+b)² + 0.5(a-b)²` — a purely volumetric error is weighted **3x** more heavily than a purely directional one, which is consistent with the original fit landing in a "match as many people as possible" corner. Fixed by dropping the redundant `total` term.
2. **Transit-gate sweep.** `max_transit_s` was originally fixed at 300s as a physical assumption (sensors ~20m apart, ~10-15s expected walking transit at a typical pace, 300s as generous slack). But no accepted match in the original run ever exceeded 156s, so the 300s ceiling wasn't protecting real matches — only widening the candidate pool feeding the "~20 equally plausible partners" ambiguity identified as the likely cause of the skew. Tested by fitting it from {60, 90, 120, 180, 300}s instead of assuming it.

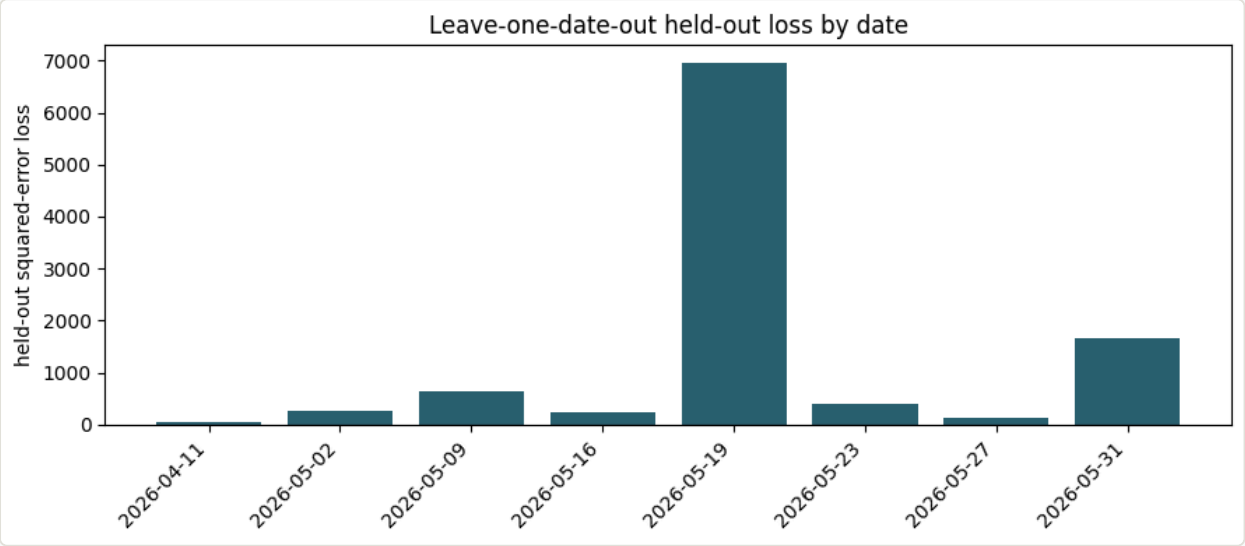
metric	before (original)	after (this revision)	read
mean held-out loss	2,205.5	1,288.9	lower, but loss function itself changed — not directly comparable
mean |A→B share error|	0.292	**0.273**	modest improvement
always-50/50 baseline	0.369	0.369	unchanged (doesn't depend on our params)
corr(pred share, manual share)	0.533	**0.490**	slightly worse
`max_transit_s`	300 (fixed)	180 (fitted)	genuine interior optimum
final applied output (all 14 dates)	5,964 matched, 3,179/2,785 split	**identical**	no real match ever needed >156s, so tightening the gate changed nothing downstream

Honest read: mixed, not a breakthrough. One metric improved modestly, another got slightly worse, and the numbers that actually ship in `travelers_matched_direction.csv` did not change at all. Both n=8 metrics are noisy enough that neither direction of movement should be over-read. The underlying bottleneck — many equally-timed candidate pairs on busy dates, which reweighting the loss or narrowing the gate cannot by itself resolve — is unchanged. Fixing *which candidates are admissible* doesn't fix *which admissible candidate is correct*; that needs either a stronger per-traveler feature or a structural constraint like order-preservation on a no-overtaking trail (see Section 10).

5.4 Leave-one-date-out results (current)

date	pred A→B	pred B→A	pred total	manual A→B	manual B→A	manual total	held-out loss

2026-04-11	30	7	37	27	1	28	45
2026-05-02	36	39	75	20	36	56	265
2026-05-09	184	81	265	181	56	237	634
2026-05-16	25	17	42	22	2	24	234
2026-05-19	64	44	108	136	2	138	6,948
2026-05-23	18	29	47	30	13	43	400
2026-05-27	11	10	21	17	0	17	136
2026-05-31	15	8	23	55	1	56	1,649



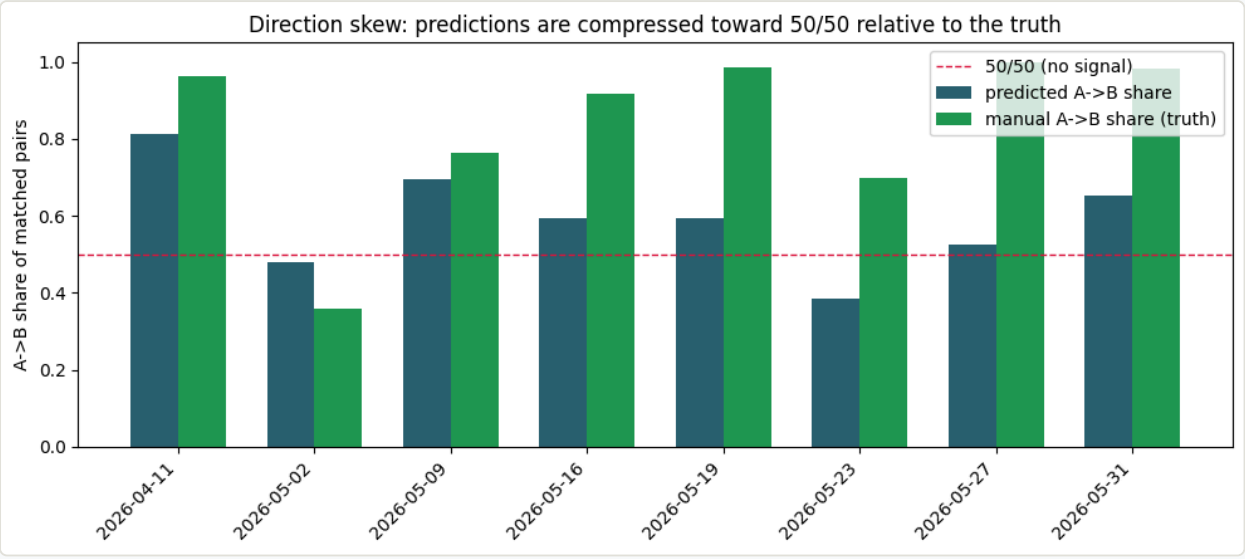
Held-out loss per date. The two structurally-unreachable dates (2026-05-19, 2026-05-31) still dominate the mean.

5.5 Direction skew — the check that actually matters

Aggregate loss can look acceptable while the direction column itself is worthless. This checks predicted vs. manual A→B *share* directly, on held-out predictions:

date	predicted A→B share	manual A→B share	error
2026-04-11	0.811	0.964	−0.153
2026-05-02	0.480	0.357	+0.123
2026-05-09	0.694	0.764	−0.070
2026-05-16	0.595	0.917	−0.322

2026-05-19	0.593	0.986	−0.393
2026-05-23	0.383	0.698	−0.315
2026-05-27	0.524	1.000	−0.476
2026-05-31	0.652	0.982	−0.330



Predicted vs. manual A→B share per date. Predictions are still compressed toward 50/50 (dashed line) relative to the truth on nearly every date.

Read honestly: still better than a coin flip, still not reliably so. Mean |share error| is 0.273 vs. a 0.369 always-50/50 baseline (a 0.096 improvement, up from 0.077). Correlation between predicted and manual share across the 8 dates is 0.490 (n=8, not statistically significant). On 2026-05-19 — 98.6% one-way in the manual log — the model now predicts 59%. On 2026-05-27 (100% one-way) it predicts 52%, unchanged.

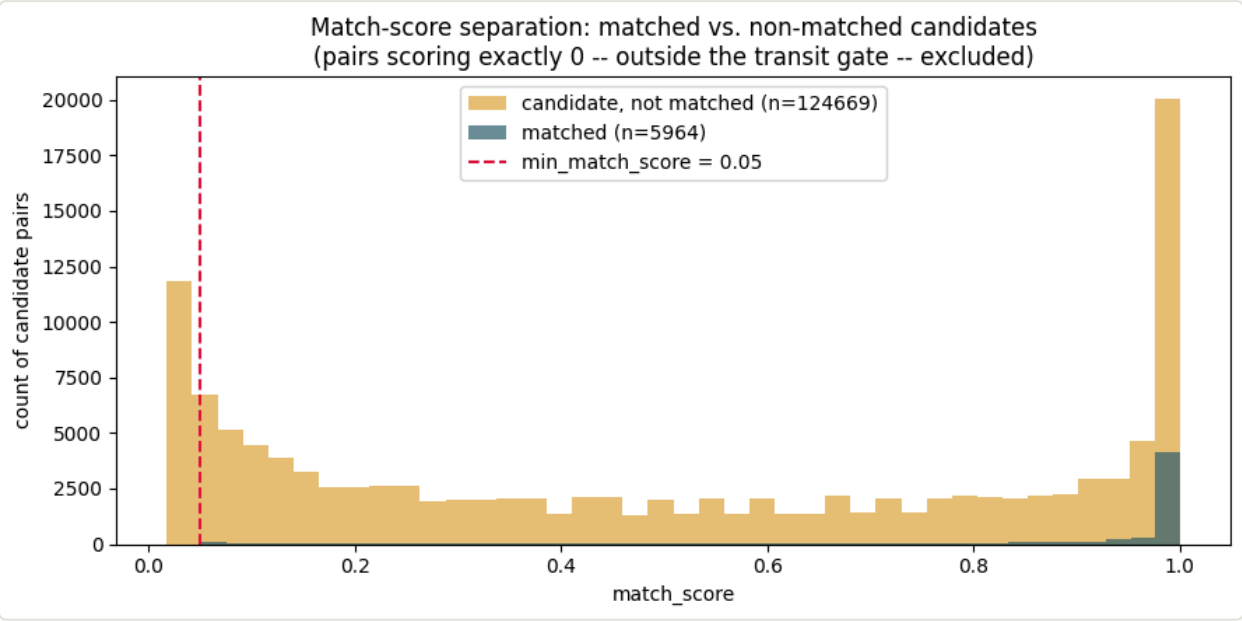
Why the fixes only moved the needle slightly: on busy dates there are still many equally-plausible candidate pairs within the transit gate — e.g. ~20 per traveler on the busiest date (2026-05-09, ~4.5 arrivals/minute/sensor). `(anchor timestamp, rssi_trend)` still cannot distinguish the true partner from an arbitrary nearby one. Both fixes changed *which* candidates the matcher is allowed to consider or how errors are weighted — neither changes the fact that, among the candidates that remain, many are still equally plausible.

7. Applying the calibrated matcher to all dual-sensor dates

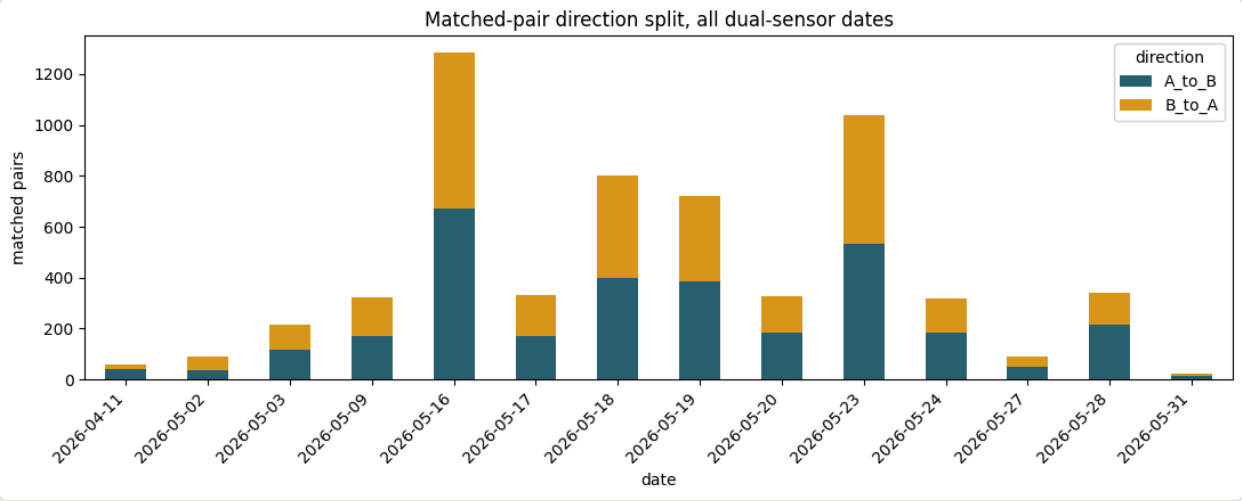
The calibrated parameters are applied to all 14 dates with data from both sensors (not just the 8 ground-truth dates).

8,010	5,964	926 / 1,120	3,179 / 2,785
total output rows	matched pairs (74.5%)	unmatched, A-only / B-only	direction: A→B / B→A (of matched)

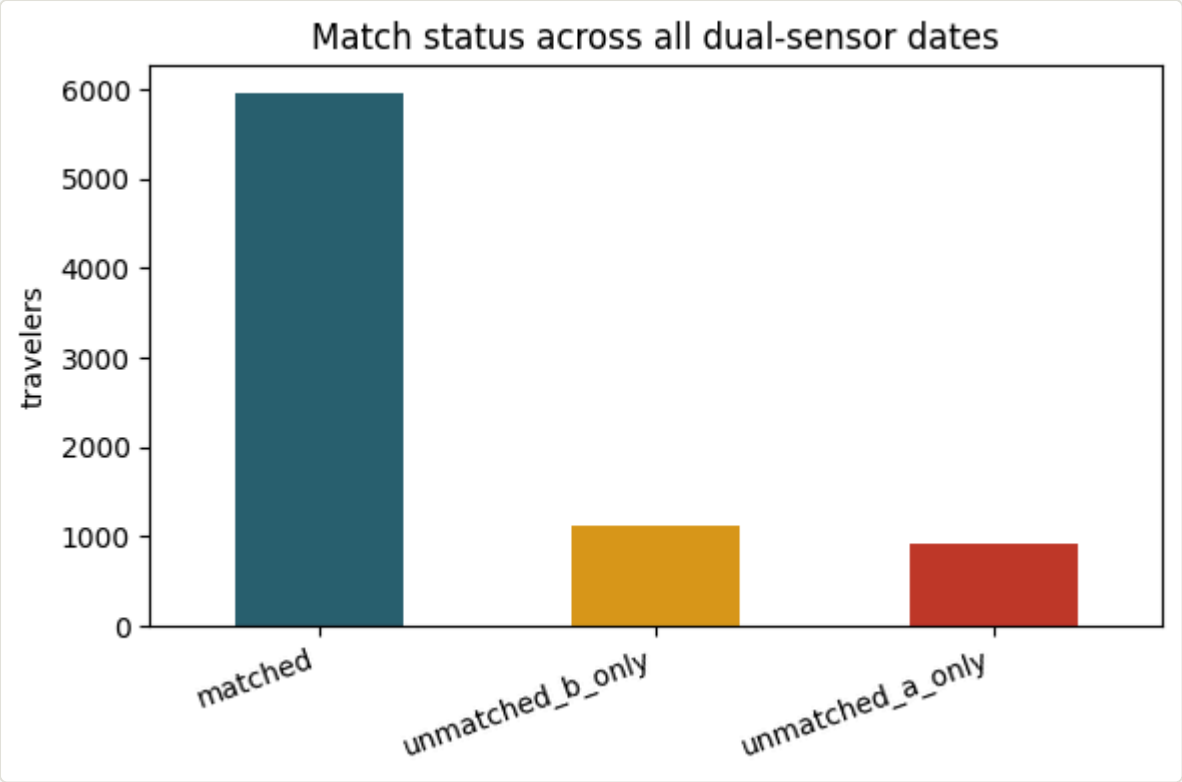
Matched-row `transit_gap_s`: mean 25.5s, median 13.0s, max 156.0s — every value physically plausible, 0 negative. The median (13.0s) lines up well with the simple physics: ~20m at a 5–6 km/h walking pace is ~10–15s — independent evidence the model is tracking real transits, not noise.



Match-score separation check (from the design's validation plan): distribution of `match_score` for accepted matches vs. candidate pairs that were considered but not matched. Pairs scoring exactly 0 (outside the now-180s transit gate) are excluded from the histogram — of the candidate pairs across all 14 dates, 5,566,561 are gated to exactly 0 (up from 5,494,887 under the old 300s gate), underscoring how much of the search space the tighter transit-time gate now eliminates before scoring even matters.



Matched-pair direction split across all 14 dual-sensor dates.



Match status totals across all dual-sensor dates: matched pairs vs. travelers seen by only one sensor.

Output schema

column	matched row	unmatched row
`date`	collection date	collection date
`match_status`	`matched`	`unmatched_a_only` / `unmatched_b_only`
`traveler_id_a`	Stage-C Sensor-A `Traveler_ID`	set for `unmatched_a_only`, else NaN
`traveler_id_b`	Stage-C Sensor-B `Traveler_ID`	set for `unmatched_b_only`, else NaN
`direction`	`A_to_B` or `B_to_A`	**NaN** — no direction is inferable
`transit_gap_s`	always in `(0, 180]` s	NaN
`match_score`	`[min_match_score, 1 + rssi_weight]`	NaN

Output: `DATA/travelers_matched_direction.csv` / `.parquet`.

8. Why two ground-truth dates are structurally unreachable

Section 5.2 showed that 2026-05-19 and 2026-05-31 have manual totals exceeding the maximum number of pairs a bipartite matching could ever produce. That alone doesn't say *why* — this section diagnoses the two dates separately and finds two **different** root causes, each checked against the raw (pre-Stage-C, pre-any-clustering) MAC counts as a falsification test: if the raw MAC count itself is already below the manual total, no clustering-threshold change could ever fix it; if the raw count is comfortably above the manual total but the Stage-C traveler count is not, the shortfall is specifically a Stage C entity-resolution effect.

date	manual total	raw MACs (A / B)	raw ceiling reachable?	Stage-C travelers (A / B)	Stage-C ceiling reachable?	compression rate (A / B)
2026-04-11	28	57 / 61	yes	42 / 42	yes	26% / 31%
2026-05-02	56	213 / 142	yes	138 / 75	yes	35% / 47%
2026-05-09	237	564 / 531	yes	265 / 268	yes	53% / 50%
2026-05-16	24	58 / 62	yes	46 / 49	yes	21% / 21%
2026-05-19	138	306 / 357	**yes**	124 / 140	**no**	**60% / 61%**
2026-05-23	43	77 / 81	yes	53 / 56	yes	31% / 31%
2026-05-27	17	37 / 36	yes	22 / 22	yes	41% / 39%
2026-05-31	56	**47 / 40**	**no**	33 / 25	**no**	30% / 38%

2026-05-19 — fixable in principle (Stage C over-merging). Raw material was sufficient: 306 raw Sensor-A MAC addresses were detected in this 76-minute window, comfortably more than the 138 people needed. But Stage C's entity resolution compressed them down to just 124 travelers — a **59-61% compression rate**, well above the 20-53% range seen on every other GT date (2026-05-09, the busiest date overall, still only compresses at 50-53%). The largest single merged cluster anywhere in the entire project sits inside this exact window: traveler `A-198-47`, 17 raw MAC addresses folded into one `Traveler_ID` (and `B-277-17` on the Sensor-B

2026-05-31 — not fixable by any Stage C or Stage D change. Even the *raw* MAC count before any clustering at all is already below the manual total: 47 on Sensor A, 40 on Sensor B, vs. 56 people counted manually. If Stage C did zero merging whatsoever (every MAC kept as its own traveler), the ceiling would still only be 40. This means people were never detected at the sensing level in the first place — a Stage A/B raw-detection gap, or a more fundamental limit of BLE-based counting: not every visitor necessarily carries an actively-advertising BLE device.

side, 19 devices). This is the already-documented "people walking together get merged" limitation from Stage C's own report, showing up unusually severely during this specific peak-traffic burst.

Decision: Stage C is left unmodified. Retuning its clustering threshold could plausibly recover some of the 2026-05-19 gap, but (1) it would not touch 2026-05-31 at all, (2) Stage C has no ground truth of its own, so validating a threshold change only against these same 8 Stage-D windows risks silently overfitting Stage C to Stage D's calibration set, and (3) any change would affect all 30 collection days' already-reported per-day traveler counts, not just this one date. Both findings are documented here as limitations rather than patched.

9. Honest assessment: does direction detection work?

What Stage D gets right:

- Matched-pair **volume** is in the right ballpark across dates.
- Every matched row's `transit_gap_s` is guaranteed physically plausible — `(0, 180]` s, zero negative-gap rows (was 18.2% in the buggy first run).
- Unmatched (single-sensor-only) travelers are reported explicitly with `direction = NaN`, not silently dropped or forced into a low-confidence match.
- Calibration methodology (leave-one-date-out, orientation re-fit per fold, explicit reachability check) does not overstate what was learned from only 8 ground-truth windows.

What Stage D does not deliver:

- **The `direction` column should not be used for per-person direction claims**, and only cautiously for aggregate skew. It is attenuated toward 50/50 and systematically understates one-way traffic.
- `match_score` is a plausibility score, not a probability — its scale changes with `rssi_weight`.
- Two of the eight ground-truth dates are structurally unreachable, for two different reasons diagnosed in Section 8 — one a Stage C over-merging artifact during peak traffic, the other a hard Stage A/B sensing ceiling — neither fixed, both documented.
- The loss-fix and transit-gate-sweep experiments (Section 6) moved one metric slightly better and one slightly worse — a real but modest effect, not a resolution of the underlying matching-ambiguity bottleneck.

10. Conclusions and recommended next steps

Stage D successfully links a substantial fraction of travelers across the two sensors — the matching mechanism (bipartite assignment, physical transit gate, forbidden-edge thresholding) is sound and produces plausible matched-pair volumes. The **direction** label built on top of that matching, however, does not clear the bar for a trustworthy signal: it beats a coin flip only modestly, is not statistically significant at n=8 ground-truth dates, and is systematically compressed toward 50/50 on exactly the dates where the true traffic is most one-directional — even after correcting the loss function's volume/direction imbalance and fitting the transit-time gate instead of assuming it.

This is the design's own anticipated outcome ("if held-out error is large or unstable across folds, that is itself a reportable finding... it would mean the feature set isn't sufficient") — and that is where the project landed, honestly reported rather than smoothed over. The two cheapest fixes have now been tried (Section 6) and shown to be insufficient on their own; what's left requires either a stronger signal or a structural assumption about the trail itself.

priority	next step	rationale
1	~~Fix the calibration loss~~ — done (Section 6), modest effect	Direction and volume now compete on equal terms in the loss; didn't resolve the skew on its own
2	~~Sweep the transit-time gate~~ — done (Section 6), modest effect	`max_transit_s` fitted to 180s (a genuine interior optimum), but final output unchanged since no real match ever needed >156s
3	Test a no-overtaking / single-file-trail assumption; if physically true, add order-preservation as a hard constraint on the matching	A structurally stronger signal than timing proximity alone — directly targets the "~20 equally plausible candidates" ambiguity that neither Section 6 fix touched
4	Consider a richer pairwise classifier with more features	Now appropriately sequenced after the cheaper fixes were shown insufficient, rather than premature
5	Stage C's 2026-05-19-style over-merging (Section 8) — diagnosed, not patched	Any fix must validate against days beyond the 8 Stage-D GT windows to avoid overfitting Stage C to Stage D's calibration set; 2026-05-31's shortfall is not addressable this way regardless

---

This report is generated from a full, current execution of `notebooks/04_stage_d_direction_linking.ipynb` against all available data, and from `docs/specs/2026-07-30-stage-d-direction-linking-design.md`'s full design and post-implementation calibration writeup. All Stage D matching, scoring and calibration logic is self-contained in that notebook. Project files, notebooks and design documents (`docs/specs/`) are in the `bluetooth_projet` project directory.